tmux build & install — Troubleshooting

Revision: 1 **Last modified:** 2026-06-29T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator troubleshooting for the build & install pipeline — rootless-Podman subuid/subgid exhaustion, the containerized-vs-native build paths, native-build C-toolchain failures (`C compiler cannot create executables`), the `install.sh` HTTPS-rewrite / private-submodule edge, and the git-ignored `.local-deps/` obtain mechanism (§11.4.77).

---

1. Overview

`bash scripts/setup.sh` (and the `curl | bash` installer that delegates to it) prefers a **hermetic containerized build** on Linux, with a **native host build** as an automatic fallback. Most install snags fall into four classes, each covered below:

#	Symptom	Section
1	Containerized build dies at image unpack with `lchown … invalid argument; potentially insufficient UIDs or GIDs available in user namespace`	§2 Rootless-Podman subuid/subgid exhaustion
2	Native build dies at `configure: error: C compiler cannot create executables` (gcc present but cannot link)	§3 → Native build fails: C compiler cannot create executables
3	"Which build path ran, and what does each need?"	§3 Containerized vs native build
4	A **private** submodule clone fails for an SSH-keyed user	§4 install.sh HTTPS-rewrite / private-submodule edge
5	"Where do libevent / ncurses / jemalloc come from, and how do I force a fresh copy?"	§5 Local dependencies (.local-deps/)

All four are **real, already-shipped behaviour** — none of the flags or commands below are invented; cross-references to the source script lines are given so you can verify each in place.

---

2. Rootless-Podman subuid/subgid exhaustion

Symptom

On a host running **rootless Podman**, the containerized build (`scripts/build_containerized.sh`, invoked by `setup.sh` step 2) fails while unpacking the base image, with an error like:

```
Error: ... lchown /etc/gshadow: invalid argument
       ... potentially insufficient UIDs or GIDs available in user
namespace
```

Root cause

Rootless Podman maps the container's user IDs into a range of **subordinate UIDs/GIDs** assigned to your host user in `/etc/subuid` and `/etc/subgid`. Unpacking a full base image needs tens of thousands of distinct IDs (it `chown`s files to many UIDs/GIDs inside the namespace). When your user's range is **missing or too small**, the unpack cannot `lchown` files such as `/etc/gshadow` and fails with the error above. This is an environment/host-ID problem, not a defect in the tmux build — the forensic anchor was a base-ALT host whose default user had no usable subuid/subgid range (`scripts/setup.sh:347-357`).

Fix (requires root, one-time)

Grant your user a 65,536-ID subordinate range and let Podman re-establish the namespace, then re-run setup:

```
# As root (or via sudo). Replace "$USER" with the build user if
        you run this for someone else.
sudo usermod --add-subuids 100000-165535 --add-subgids
        100000-165535 "$USER"

# Stop the user's containers + kill its pause process so the new
        ranges take effect.
podman system migrate

# Re-run the build (rootless, no sudo).
bash scripts/setup.sh --rebuild
```

`setup.sh` itself prints this exact `usermod` + `podman system migrate` remedy when the containerized build fails (`scripts/setup.sh:362-367`).

Verify the fix

```
# Your user now has a subordinate range in BOTH files (USERNAME:START:COUNT):
grep "^$(id -un):" /etc/subuid /etc/subgid
#   you:100000:65536        (the COUNT — 65536 — must be large enough)

# A trivial rootless run no longer errors on the namespace map:
podman unshare cat /proc/self/uid_map
```

If `grep` shows no line for your user (or a tiny count), the range was never added — re-run the `usermod` step. The 65,536-ID range is the size the Podman rootless documentation recommends (see §6 Sources verified).

Don't want to touch root?

You don't have to. If the host already has a C toolchain, `setup.sh` **automatically falls back to the native build** when the container build fails (see §3) — so a subuid problem need not block you at all.

---

3. Containerized vs native build

The project produces the **same** tmux binary two ways. `setup.sh` chooses automatically; understanding both helps when one path is unavailable.

Containerized (hermetic) — the default on Linux

- Driver: `scripts/build_containerized.sh`, run by `setup.sh` step 2 when a container engine (podman or docker) is detected.
- Builds inside an isolated container cgroup (host insulated, §12.9) with `--network none`; the build dependencies (libevent / ncurses / jemalloc) live **inside the image**, so the host needs no `-dev` packages.
- Needs: a working **podman or docker** (rootless Podman needs the subuid ranges from §2).

Native (host) — the fallback

- Driver: `scripts/build_native.sh`.

- `setup.sh` runs it when **no container engine is present**, OR when the containerized build **fails** for a host reason (e.g. the subuid exhaustion above, or no network to pull the base image) — the fallback is automatic and prints why it triggered (`scripts/setup.sh:346-372`, §11.4.101 reversible decision).
- Needs on the host: a **C compiler + libevent + ncurses (widec)** development headers, plus `autoconf` / `automake` / `pkg-config` / `bison`.
  - On Linux you can install these once with `sudo bash scripts/install_deps.sh` (provides `libevent-dev` + `libncurses-dev` + the autotools), or
  - rely on the git-ignored `.local-deps/` obtain mechanism, which builds local copies of libevent + ncurses when the host lacks the `-dev` packages (see §5). On a minimal host (e.g. one with no `libevent-dev` at all) this is what lets the native fallback link successfully.

Both paths produce a tmux binary that the verification gate (`scripts/verify.sh`) must pass before `tmx` is exported to your PATH — neither path bypasses the §11.4 GREEN gate.

Native build fails: `C compiler cannot create executables`

Symptom

The native build aborts at the very start of tmux's `./configure` probe (driven by `scripts/build_native.sh`) with:

```
configure: error: C compiler cannot create executables
See `config.log' for more details
```

`gcc` (or `clang`) is on your `PATH` — `gcc --version` prints a version — yet `configure`'s first compile-**and-link** probe still fails. (`setup.sh` surfaces this dependency need — `scripts/setup.sh:235,366`; this section is the fuller, by-hand version.)

Root cause

FACT (verified on an **ALT Linux 11** host — the operator's target distro, same as the dev host — with `rpm -qf /usr/lib64/crt1.o` → `glibc-devel`): on ALT Linux the "C compiler cannot create executables" error means **`glibc-devel` is not installed** — it supplies the C-library startup objects (`/usr/lib64/crt1.o`, `crti.o`, `crtn.o`) the linker needs, so `gcc` compiles but cannot link. The same class applies on every distro: a C *compiler* binary alone cannot link an executable — it also needs the C-library development objects + the `ld` linker (`binutils`). The package that supplies them is **`glibc-devel`** on ALT / Fedora / RHEL, and **`libc6-dev`** on Debian/

Ubuntu (pulled by the `build-essential` meta-package); on macOS the **Xcode Command Line Tools** provide `clang` + the macOS SDK + the linker. See §6 Sources verified.

Fix (per OS — §11.4.81 cross-platform; ALT Linux is the primary target)

`scripts/setup.sh` surfaces these build dependencies, and the project's `scripts/install_deps.sh` installs them automatically when run with the privilege to do so — it detects the host package manager (apt-rpm / apt-get / dnf / pacman / zypper / apk / Homebrew) and installs the C toolchain + libevent + ncurses + jemalloc + autotools (`scripts/install_deps.sh:55-117`):

```
# One-shot, all distros. ALT Linux uses root (no sudo); macOS uses
        Homebrew, no root.
bash scripts/install_deps.sh        # run as root on Linux
```

The per-distro commands below are the **by-hand fallback** for when the automatic install cannot run — no root privilege, or an unrecognised distro.

ALT Linux 11 (apt-rpm) — the operator's target host; run as root (ALT uses root, not `sudo`). All package names verified present in the apt-rpm cache on an ALT 11 host:

```
# ALT Linux (apt-rpm) — PRIMARY.
# libncursesw-devel Depends: libncurses-devel, so apt-rpm pulls
        the widec
# ncurses header + libncursesw.so + ncursesw.pc automatically.
apt-get install -y gcc glibc-devel make libevent-devel
        libncursesw-devel \
                    autoconf automake pkg-config bison flex
```

Additional variants:

```
# Debian / Ubuntu (apt / dpkg):
sudo apt-get install -y build-essential libevent-dev libncurses-
        dev \
                        autoconf automake pkg-config bison

# Fedora / RHEL / CentOS (dnf):
sudo dnf groupinstall -y "Development Tools" \
  && sudo dnf install -y libevent-devel ncurses-devel \
                         autoconf automake pkgconf-pkg-config
        bison

# macOS (installs clang + the macOS SDK + ld):
xcode-select --install
```

Verify the fix

Prove the toolchain can now compile **and link** a trivial executable before re-running setup:

```
echo 'int main(){return 0;}' | gcc -x c -o /tmp/cc-test - && echo
        "C toolchain OK"
```

`C toolchain OK` means `configure` will get past the probe — re-run `bash scripts/setup.sh`. On macOS substitute `clang` for `gcc` (the Command Line Tools alias `gcc` to `clang`, so the same line also works). If linking still fails, open `config.log` in the build directory — autoconf records the exact missing object / linker error there.

---

4. install.sh HTTPS-rewrite / private-submodule edge

Background

The `curl | bash` installer clones over **HTTPS by default** so a fresh user **without SSH keys** can still fetch the project and its **public** github submodules. Because the submodules pin SSH URLs in `.gitmodules`, the installer applies a git `insteadOf` rewrite by default — `url.https://github.com/.insteadOf=git@github.com:` — so a keyless user's recursive clone resolves those public submodules over HTTPS (`scripts/install.sh:142-152`).

The edge

If you **do** have SSH keys configured **and** need a **private** submodule, the default HTTPS rewrite works against you: it rewrites the submodule's `git@github.com:` SSH URL to `https://github.com/…`, which then attempts an **unauthenticated** HTTPS fetch (no credential) and the private clone fails. The honest boundary is documented in the script itself: the HTTPS rewrite does **not** grant access to private submodules (`scripts/install.sh:144-146`).

Recovery

Disable the rewrite so your SSH key is used for the private submodule. Either:

```
# Env var (works under the curl|bash pipe):
TMX_INSTALL_NO_HTTPS_REWRITE=1 \
  curl -fsSL https://raw.githubusercontent.com/vasic-digital/tmux/
        main/scripts/install.sh | bash
```

```
# …or the equivalent flag when running a downloaded copy:
bash install.sh --no-https-rewrite
```

With the rewrite disabled, the submodules clone via their pinned `git@github.com:` SSH URLs and your key authenticates the private fetch (`scripts/install.sh:34,82,92,150-151`). Public-submodule keyless users should **not** set this — they need the default rewrite.

See also the "Private submodule with no access" / "Edge cases" notes in `../scripts/install.md`.

---

5. Local dependencies (`.local-deps/`)

What it does

`scripts/obtain_local_deps.sh` provides the project's build/runtime dependencies **git-ignored, per host** (§11.4.77), so a fresh clone works out-of-the-box even on a minimal host. `setup.sh` step 1b invokes it for three deps:

- **jemalloc** — the runtime hardening allocator (preloaded by the `tmx` wrapper); a container-built binary's `DT_NEEDED libjemalloc.so.2` must resolve at runtime.
- **libevent + ncurses (widec)** — the tmux **build** dependencies the **native** fallback (§3) needs when the host lacks `libevent-dev` / `libncurses-dev`.

For each dep the script first **resolves** an already-present copy by absolute path (§11.4.111 — never ambient `PATH`), and only **obtains** (source-builds or extracts) a local copy when the dep is genuinely missing. A present, valid local copy is reused, not rebuilt (idempotent).

Where it lands

```
.local-deps/<uname-s>_<uname-m>/lib/<libname>     the obtained library
.local-deps/<uname-s>_<uname-m>/resolved.env      sourceable KEY=VALUE paths
```

`resolved.env` exposes the resolved paths — `JEMALLOC_SO` / `JEMALLOC_LIBDIR`, `LIBEVENT_LIBDIR` / `LIBEVENT_INCDIR`, `NCURSES_LIBDIR` / `NCURSES_INCDIR` (each with a `*_SOURCE` provenance tag). `build_native.sh`, `verify.sh`, `run_all.sh`, and the `tmx` wrapper all consume it (`scripts/obtain_local_deps.sh:49-55`).

Force a fresh local copy

To skip host detection and always source-build the deps into the local prefix (useful when a host copy is broken, or to reproduce the minimal-host path):

```
# Force-obtain all three locally (skips host resolution):
FORCE_OBTAIN=1 DEPS="jemalloc libevent ncurses" bash scripts/
        obtain_local_deps.sh

# Override the git-ignored root if you need it elsewhere:
LOCAL_DEPS_ROOT=/some/path FORCE_OBTAIN=1 bash scripts/
        obtain_local_deps.sh
```

`FORCE_OBTAIN=1`, `DEPS=…`, `LOCAL_DEPS_ROOT=…`, and `OBTAIN_METHOD=auto|source|container` are the real env inputs (`scripts/obtain_local_deps.sh:34-47`). The obtain step needs a C compiler + make + curl/wget + tar + a sha256 tool for the source method; on Linux it can also use the container image as a fallback obtain method. No `sudo` is run.

Full reference: `../scripts/obtain_local_deps.md`.

---

6. Sources verified 2026-06-29

- **Rootless Podman subuid/subgid + `usermod --add-subuids/--add-subgids` + `podman system migrate`** — Podman *Basic Setup and Use of Podman in a Rootless environment* tutorial: https://github.com/containers/podman/blob/main/docs/tutorials/rootless_tutorial.md (fetched 2026-06-29). It documents the exact `usermod --add-subuids 100000-165535 --add-subgids 100000-165535 <user>` command (a 65,536-ID range) and the requirement to run `podman system migrate` after changing `/etc/subuid` / `/etc/subgid` so the new namespace mappings take effect.
- **`/etc/subuid` / `/etc/subgid` file format** (`USERNAME:UID:COUNT`) — the same Podman tutorial; offline-authoritative man pages `man 5 subuid` / `man 5 subgid` and `man 8 usermod` (the `--add-subuids` / `--add-subgids` options) corroborate the syntax. `man podman-system-migrate` is the offline authority for the migrate step.
- **Podman troubleshooting — `rootless setup user: invalid argument` (Section 10)** — Podman *Troubleshooting* guide: https://github.com/containers/podman/blob/main/troubleshooting.md (fetched 2026-06-29). Re-confirms the `/etc/subuid` + `/etc/subgid` `USERNAME:UID:RANGE` format (`johndoe:100000:65536`), the `usermod --add-subuids … --add-subgids …` alternative, the requirement that each user's range be **unique / non-overlapping**, that the range must **cover all UIDs the container requires**, and that `podman system migrate` stops the containers + kills the pause process so the new

mapping takes effect. (The `lchown /etc/gshadow: invalid argument` unpack failure is this exhaustion class — a `chown` inside the namespace hitting the end of an undersized/absent range.)

- **Native build `C compiler cannot create executables` → ALT `glibc-devel` (primary), Debian `build-essential/libc6-dev` (variant)** — the ALT Linux package names were verified directly on an **ALT Linux 11** host (the operator's target distro), NOT guessed (§11.4.6): `rpm -qf /usr/lib64/crt1.o` → `glibc-devel-2.40.0.224.573a-alt1` (proves `glibc-devel` supplies the startup objects whose absence yields the error); `apt-cache depends libncursesw-devel` → `Depends: libncurses-devel` (proves the single `libncursesw-devel` pulls the widec ncurses header + `libncursesw.so` + `ncursesw.pc` automatically — `pkg-config --libs ncursesw` → `-lncursesw -ltinfo`); and `apt-cache show` resolves all eleven packages (`gcc glibc-devel make libevent-devel libncursesw-devel autoconf automake pkg-config bison flex`) in the apt-rpm cache. The project's own `scripts/install_deps.sh` (the per-distro dependency list, lines 36-80) is the secondary project source. The cross-distro root cause (a present `gcc` cannot link without the C-library startup objects `Scrt1.o/crti.o/crtn.o`
  - `ld` from `binutils` — `libc6-dev` on Debian, pulled by `build-essential`) is corroborated by the community consensus on this exact autoconf error (https://www.linuxquestions.org/questions/slackware-14/configure-error-c-compiler-cannot-create-executables-4175698040/, https://forums.linuxmint.com/viewtopic.php?t=346339, fetched 2026-06-29) and the offline-authoritative `apt show build-essential` / `man gcc`. The Fedora `"Development Tools"` group and macOS `xcode-select --install` are the standard, long-stable foundational toolchain bootstraps for those platforms.
- **The build-pipeline behaviours** (containerized-vs-native fallback, `TMX_INSTALL_NO_HTTPS_REWRITE`, `.local-deps/` obtain) are verified against the shipped sources `scripts/setup.sh`, `scripts/install.sh`, `scripts/install_deps.sh`, `scripts/build_native.sh`, and `scripts/obtain_local_deps.sh` at tag `tmux-1.0.30` (line references inline above).